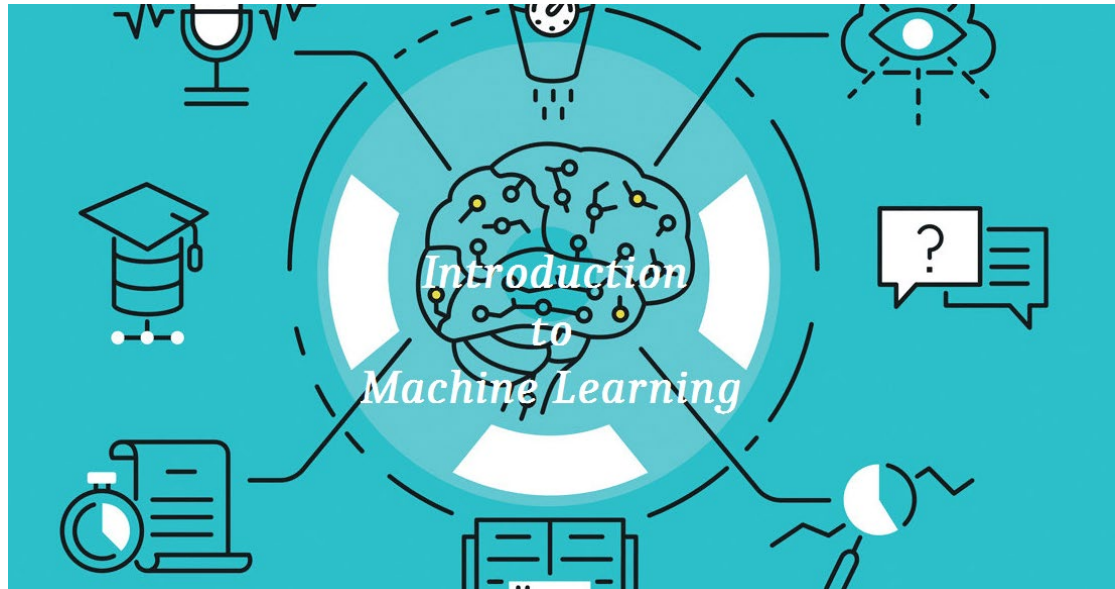


機器學習課輔

Machine Learning Tutoring



製作人：甯文駿

3-11 範例.ipynb

連結：<https://github.com/ningmao47/Machine-Learning-Tutoring/blob/main/3-11%E7%AF%84%E4%BE%8B.ipynb>

主要內容是關於 Python 資料分析與遺漏值處理 (Missing Value Handling) 的基礎練習，使用的核心資料集是機器學習中經典的 鳶尾花資料集 (Iris Dataset)。

以下是該檔案程式碼邏輯的詳細解釋：

1. 載入資料 (Loading Data)

程式首先導入了必要的函式庫，包括資料處理常用的 pandas、numpy 以及 sklearn 的資料集模組。

- **程式碼邏輯：**使用 `load_iris()` 函數將鳶尾花資料載入到變數 `iris` 中。
- **資料整理：**透過 `pd.DataFrame` 將原始資料轉換成表格格式 (DataFrame)，並將特徵名稱設為欄位名稱，最後加入一欄 `target` 作為分類標籤。該資料集共有 150 筆觀測值與 5 個欄位。

2. 遺漏值檢查 (Checking for Missing Values)

在進行模型訓練前，確認資料的完整性是關鍵步驟。

- **程式碼邏輯：**
 - 使用 `df.info()` 查看資料框的結構、每欄的非空值數量以及資料型態。
 - 使用 `df.isnull().sum()` 統計各個欄位中遺漏值 (NaN) 的總數。根據輸出結果，此資料集在所有欄位（如花萼長寬、花瓣長寬）中皆無遺漏值。

3. 遺漏值視覺化 (Visualizing Missing Values)

檔案最後使用了 `missingno` 函式庫來進行視覺化分析。

- **程式碼邏輯：**調用 `msno.bar(df)` 繪製長條圖。

- **目的：**即使數值統計顯示沒有遺漏值，透過視覺化圖表可以更直觀地確認資料的完整性。在此範例中，長條圖會顯示所有欄位的完整度皆為100%。

總結：

此檔案展示了一個標準的資料科學工作流程初步階段：從**載入原始資料**、**轉換為結構化表格**到**檢查並視覺化資料完整性**。這對於確保後續機器學習模型的準確性至關重要。

3-25 範例.ipynb

連結：<https://github.com/ningmao47/Machine-Learning-Tutoring/blob/main/3-25%E7%AF%84%E4%BE%8B.ipynb>

1. 資料載入與初步探索

- **載入資料**：使用 `sklearn.datasets.load_iris()` 讀取資料，並轉換為 `pandas` 的 `DataFrame` 格式。
- **遺漏值檢查**：利用 `df.info()` 和 `df.isnull().sum()` 確認資料完整性（`Iris` 資料集通常是完整的，沒有缺失值）。
- **敘述性統計**：使用 `df.describe()` 查看各特徵（如花萼、花瓣的長寬）的平均值、標準差與四分位數。

2. 資料視覺化

程式碼使用了 `matplotlib` 和 `seaborn` 進行探索式資料分析（EDA）：

- **直方圖 (Histogram)**：觀察各個特徵的數值分佈情況。
- **箱型圖 (Boxplot)**：比較不同品種（`Target`）在各項生理特徵上的差異。
- **熱力圖 (Heatmap)**：計算特徵間的相關係數（`Correlation`），觀察哪些特徵（如花瓣長度與寬度）與分類目標有最強的關聯性。

3. 資料預處理

- **標籤編碼 (Label Encoding)**：雖然 `Iris` 的 `target` 已經是數值，但程式碼演示了如何使用 `LabelEncoder` 將類別資料轉換為整數。
- **資料切割**：使用 `train_test_split` 將資料分為 **50% 訓練集**與 **50% 測試集**，並設定 `stratify=y` 以確保訓練與測試集中的類別比例一致。
- **特徵縮放 (Feature Scaling)**：演示了 `StandardScaler`（標準化），將數據轉換為平均值為 0、標準差為 1 的分佈，這對於許多機器學習演算法（如邏輯迴歸）非常重要。

4. 模型建立與評估

程式碼實作了兩種常見的分類模型：

- **決策樹 (Decision Tree)：**
 - 直接使用所有特徵進行訓練。
 - 提供了一個自定義函數 `evaluate_model`，輸出 **混淆矩陣 (Confusion Matrix)**、**分類報告 (F1-score, Recall, Precision)** 以及 **AUC 曲線值**。
- **邏輯迴歸 (Logistic Regression)：**
 - 為了視覺化，這部分特別只選取了兩個特徵：`petal width` (花瓣寬度) 與 `sepal width` (花萼寬度)。
 - **決策邊界 (Decision Boundary)**：程式碼畫出了模型在二維空間中是如何劃分三個品種的界線。

總結：

3-25 範例 比 3_11 完整得多。這份範例涵蓋了從 **Clean Data (資料清洗)**、**Visualization (視覺化)**、**Preprocessing (預處理)** 到 **Model Evaluation (模型評估)** 的標準作業流程。

4-08 範例.ipynb

連結：<https://github.com/ningmao47/Machine-Learning-Tutoring/blob/main/4-08%E7%AF%84%E4%BE%8B.ipynb>

1. 決策樹的視覺化 (Decision Tree Visualization)

除了訓練模型外，這份程式碼利用 `sklearn.tree.plot_tree` 將決策樹的邏輯結構圖形化：

- **節點分析**：你可以直觀看到模型是如何根據特徵（如 `petal length`）的數值進行分支。
- **分類準則**：每個節點顯示了 Gini 系數（不純度）、樣本數以及該節點最終預測的類別。

2. 隨機森林演算法 (Random Forest Classifier)

這是本範例最重要的進階部分。隨機森林是一種**集成學習 (Ensemble Learning)**方法：

- **多樹模型**：它由多棵決策樹組成，透過「多數決」的方式產生最終預測結果，通常比單一決策樹更穩定且不易過度擬合 (Overfitting)。
- **效能評估**：程式碼計算了隨機森林在訓練集與測試集上的準確度 (Accuracy Score)。

3. 決策邊界對比 (Decision Boundary Comparison)

程式碼特別針對單一決策樹與隨機森林畫出了決策邊界圖：

- **選取特徵**：同樣選取 `petal width` 與 `sepal width` 作為繪圖基準。
- **邊界差異**：你可以觀察到，隨機森林的決策邊界通常比單一決策樹更為複雜或平滑，展現了集成模型處理資料空間的方式。

4. 完整的評估流程

延續了之前的優良做法，程式碼包含了：

- **混淆矩陣 (Confusion Matrix)**：觀察模型在各品種間的誤判情況。

- **分類報告 (Classification Report)**：包含 Precision、Recall 與 F1-score。
- **AUC 評分**：使用 `roc_auc_score` 並設定 `multi_class='ovr'` (One-vs-Rest) 來處理多分類問題的 AUC 計算。

總結

這份 4/08 的範例旨在讓學習者理解從**簡單模型（決策樹）**過渡到**複雜集成模型（隨機森林）**的過程，並透過視覺化手段（樹狀圖與邊界圖）深入理解機器學習模型的運作原理。

4-22 範例.ipynb

連結：<https://github.com/ningmao47/Machine-Learning-Tutoring/blob/main/4-22%E7%AF%84%E4%BE%8B.ipynb>

這份 4/22 的範例程式碼進入了**特徵工程 (Feature Engineering)** 與**多模型比較**的進階階段。延續之前的 Iris 資料集，本篇重點在於如何篩選最重要的特徵，並評估不同演算法在相同資料下的表現。

以下是該檔案程式碼邏輯的詳細解釋：

1. 特徵重要性 (Feature Importances)

程式碼利用隨機森林 (Random Forest) 的內建屬性 `feature_importances_` 來評估各個特徵對分類的貢獻度：

- **視覺化排序**：計算後使用長條圖將特徵按重要性由高到低排列。
- **關鍵發現**：在 Iris 資料集中，通常 petal width 和 petal length 的重要性遠高於花萼 (sepal) 相關特徵。

2. 特徵篩選與優化

這份範例實作了減少維度的策略：

- **前 50% 篩選**：程式碼自動選取重要性最高的前兩名特徵 (Top 50%)，並僅使用這些特徵重新訓練模型。
- **效能觀察**：註解中提到，經過特徵篩選後，測試結果有些微提升，這說明了剔除雜訊特徵 (Noise) 能幫助模型更專注於關鍵資訊。

3. 多演算法橫向對比

程式碼同時訓練並評估了三種經典的機器學習模型，這對於理解不同演算法的「個性」非常有幫助：

- **邏輯迴歸 (Logistic Regression)**：線性的分類器。
- **支持向量機 (SVM)**：擅長尋找最大間隔的邊界，此處啟用了 `probability=True`。
- **K-近鄰演算法 (KNN)**：基於空間距離進行分類。

4. 決策邊界視覺化 (Decision Boundary Comparison)

為了直觀比較上述模型的差異，程式碼定義了 `plot_decision_boundary` 函數，分別畫出三種模型的決策邊界圖：

- LR 展現出較為僵硬的線性劃分。
- SVM 與 KNN 則能產生更具彈性的非線性邊界，來包裹不同類別的資料點。

5. 評估標準一致化

所有模型都統一通過 `evaluate_model` 函數進行測試，包含：

- AUC 評分：使用 `multi_class='ovr'` 處理多分類的曲線下面積。
- 混淆矩陣：比較訓練集與測試集的預測準確度，檢查是否有過度擬合 (Overfitting) 的現象。

總結

這份 4/22 的範例展示了「特徵選取 -> 模型訓練 -> 視覺化對比」，特別強調了特徵品質對於模型效能的影響。

4-29 範例.ipynb

連結：<https://github.com/ningmao47/Machine-Learning-Tutoring/blob/main/4-29%E7%AF%84%E4%BE%8B.ipynb>

4/29 的範例程式碼將重點轉移到了降維技術 (Dimensionality Reduction)，這是處理高維度資料時非常關鍵的步驟。程式碼比較了兩種最常用的降維方法：PCA (主成分分析) 與 LDA (線性判別分析)。

以下是該檔案程式碼邏輯的詳細解釋：

1. 降維技術對比 (PCA vs. LDA)

程式碼同時實作並視覺化了兩種邏輯完全不同的降維方式：

- **PCA (主成分分析)：**
 - **目標：**尋找能保有原始資料最大變異量 (Variance) 的方向。
 - **特性：**屬於**非監督式學習**，不考慮標籤 (y)，單純從特徵分佈找主成分。
 - **發現：**在前兩個主成分 (PC1, PC2) 就已經保留了約 **95.8%** 的原始資訊，顯示降維效果極佳。
- **LDA (線性判別分析)：**
 - **目標：**尋找能讓「類別間距離最大化」且「類別內差異最小化」的方向。
 - **特性：**屬於**監督式學習**，會利用標籤資訊來優化分類邊界。
 - **發現：**LD1 幾乎貢獻了所有的分類資訊，這表示在 LDA 的維度下，只需一個軸就能有效區分 Iris 的三個品種。

2. 解釋變異數與累積貢獻度

程式碼利用柱狀圖與折線圖展示了降維後的資訊保留量：

- **Explained Variance：**顯示每個分量分別解釋了多少比例的變異。
- **Cumulative Variance：**顯示累積前 \$n\$ 個分量後的總資訊量，用來決

定要保留幾個維度。

3. 降維後的模型表現

在決定保留前 2 個維度後，程式碼使用了 Logistic Regression 與 SVM 進行建模，並得出以下準確度 (Accuracy)：

- **PCA 組合**：將資料壓縮到不考慮標籤的 2D 空間後再分類。
- **LDA 組合**：將資料壓縮到最具分類代表性的 2D 空間後再分類。
- **結果**：通常 LDA 降維後的分類準確度會非常高，因為其降維過程本身就是為了「好分類」而設計的。

4. 決策邊界視覺化

最後，程式碼畫出了 2x2 的對比圖，展示了在 PCA 空間與 LDA 空間中，不同模型劃分資料的方式：

- **PCA 空間**：點的分佈較為自然擴散。
- **LDA 空間**：點的聚集度更高，類別間的界線更為清晰。

總結與後續方向

這份範例總結了三種特徵處理的方式供你比較：

1. **原始資料**：保留所有特徵。
2. **特徵選取**：根據重要性 (Importance) 挑選部分原始特徵 (如 4/22 範例)。
3. **降維技術**：將原始特徵轉換、投影到全新的低維度空間 (如本次 4/29 範例)。

5-06 範例.ipynb

連結：<https://github.com/ningmao47/Machine-Learning-Tutoring/blob/main/5-06%E7%AF%84%E4%BE%8B.ipynb>

5/08 的範例程式碼是系列教學的**集大成之作**。它不再只是單純的訓練模型，而是導入了工業界標準的**模型選擇 (Model Selection)** 與 **超參數調優 (Hyperparameter Tuning)** 流程。

以下是該檔案程式碼邏輯的詳細解釋：

1. K-折交叉驗證 (K-Fold Cross-Validation)

為了避免「資料切割」偶然性導致的評估偏差，程式碼引入了 StratifiedKFold：

- **運作原理**：將資料分成 5 份 (Folds)，輪流拿其中 1 份當驗證集，其餘 4 份當訓練集。
- **分層抽樣 (Stratified)**：確保每一折中各類別的比例與原始資料一致，這在處理分類問題時至關重要。
- **詳細指標**：程式碼不僅計算了 Accuracy，還記錄了每一折的 Precision、Recall 與 F1-score。

2. 網格搜索 (Grid Search CV)

程式碼針對不同模型定義了參數網格 (Param Grid)：

- **隨機森林 (RF)**：調整樹的數量 (n_estimators) 與最大深度 (max_depth)。
- **支持向量機 (SVM)**：尋找最佳的懲罰係數 (C) 與核函數 (kernel)。
- **K-近鄰 (KNN)**：優化鄰居數 (n_neighbors)。
- **自動篩選**：GridSearchCV 會跑完所有組合，告訴你哪一組參數的平均表現 (Best Score) 最好。

3. 多模型橫向綜合評估

程式碼一次跑完五種模型：LR、RF、SVM、DT、KNN。

- **Pipeline 概念**：雖然程式碼中段手動做了標準化，但在註解部分展示了使用 `make_pipeline` 將 `StandardScaler` 與模型封裝在一起的專業寫法，這能有效防止「資料洩漏 (Data Leakage)」。
- **AUC 多分類處理**：延續使用 `multi_class='ovr'` 來評估模型在多類別下的分辨能力。

總結：學習路徑回顧

從 3/25 到 5/06，你的學習路徑展示了完整的資料科學進程：

1. **基礎篇**：載入、清洗、簡單建模 (3/25)。
2. **視覺化篇**：決策樹與邊界視覺化 (4/08)。
3. **特徵篇**：特徵重要性與篩選 (4/22)。
4. **降維篇**：PCA 與 LDA 空間轉換 (4/29)。
5. **優化篇**：交叉驗證、網格搜索與自動化評報表 (5/06)。

5-27 範例.ipynb

連結：<https://github.com/ningmao47/Machine-Learning-Tutoring/blob/main/5-27%E7%AF%84%E4%BE%8B.ipynb>

1. 集成學習 (Ensemble Methods)

這份程式碼一次實作了機器學習領域中最主流的幾種集成技術，旨在透過結合多個弱學習器來構建一個強學習器：

- **Bagging (裝袋法)**：透過對資料進行隨機取樣 (Bootstrap) 來訓練多個獨立模型並取其平均。
- **Random Forest (隨機森林)**：Bagging 的進化版，除了樣本隨機，還加入了特徵隨機，是目前最穩定的模型之一。
- **AdaBoost (權重調整法)**：屬於 Boosting 技術，會根據前一個模型的錯誤來調整樣本權重，讓後續模型專注於難分類的樣本。
- **XGBoost (極端梯度提升)**：現代資料科學比賽 (如 Kaggle) 的常勝軍，以效能極高和精確度準確著稱。

2. 深度評估指標：特異度 (Specificity)

與之前的範例不同，這份程式碼手動實作了 `specificity_score` 函數。

- **定義**：特異度衡量的是模型「正確識別負樣本」的能力 (即 $TN / (TN + FP)$)。
- **重要性**：在醫療診斷等領域，正確排除非患病者 (高特異度) 與正確找出患病者 (高敏感度/召回率) 同等重要。

3. 多分類 ROC 曲線視覺化

程式碼利用 `label_binarize` 將多分類問題轉化為二元對立，並繪製出完整的 ROC 曲線圖：

- **One-vs-Rest (OvR)**：分別畫出三個類別 (Class 0, 1, 2) 的 ROC 曲線，觀察模型對特定品種的辨識能力。
- **Micro-Average (微平均)**：將所有類別的預測結果匯總後計算的平均

AUC，代表模型的整體表現。

- **圖形意義**：曲線越靠近左上角，代表 True Positive Rate 越高且 False Positive Rate 越低，模型效能越好。

4. 模型效能對比

透過 `evaluate_model` 函數，你可以清楚觀察到：

- **決策樹 vs 集成模型**：通常單一決策樹容易過度擬合 (Overfitting)，而隨機森林或 XGBoost 在測試集 (Test Set) 上的表現往往更穩健。
- **三合一視覺化圖表**：每個模型都會輸出「訓練集混淆矩陣」、「測試集混淆矩陣」以及「ROC 曲線」，提供全方位的診斷。